



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO
FACULTAD DE INGENIERÍA

Proyecto Integral de Ingeniería de Software

Tablero Kanban del Proyecto

Sistema Inteligente para la Gestión y Priorización de Incidencias de Soporte Técnico (SIGPI)

Versión 2 · derivada de las historias de usuario · React + TypeScript · FastAPI en capas

5 sprints · Scrum + RUP · 4 integrantes

EQUIPO

Haide Sánchez Gutiérrez · Laura Valeria Rodríguez Morales
Jesús Iniesta Valverde · Emmanuel Bernal Espejel

DOCENTE

Silvia Edith Albarrán Trujillo

METODOLOGÍA

Modelo de proceso: RUP (Proceso Unificado) | Seguimiento: Scrum

Toluca de Lerdo, Estado de México · septiembre de 2026

Contenido

Nota de rediseño	3
1. Marco metodológico	4
2. Justificación de la metodología elegida	4
3. Arquitectura de referencia: React + TypeScript y FastAPI en capas	6
4. Leyenda y convenciones del tablero	7
5. Distribución de actividades por integrante	8
6. Tablero Kanban por sprint	9
Sprint 1 · RUP: Inicio	9
Sprint 2 · RUP: Elaboración	10
Sprint 3 · RUP: Construcción I	12
Sprint 4 · RUP: Construcción II	13
Sprint 5 · RUP: Transición	15
7. Resumen de actividades por sprint	17
8. Trazabilidad: historias de usuario y tarjetas	17
9. Backlog posterior (entregas 2 y 3)	19
Referencias	21

Nota de rediseño

Esta versión recrea el tablero Kanban del proyecto a partir de las historias de usuario (versión 3), con el stack React + TypeScript en el frontend y FastAPI con arquitectura en capas en el backend. Sustituye al tablero anterior, cuyas tarjetas se habían planeado por fase de RUP sin derivarse de las historias.

Aspecto	Tablero anterior	Este tablero
Origen de las tarjetas	Fase de RUP y casos de uso	Actividades de las 41 historias de usuario (145 actividades); el tablero programa las de las 18 historias «Debe» (69 actividades)
Tecnología	FastAPI + React, sin capas explícitas	FastAPI en capas (api, schemas, services, domain, repositories, models, integrations, core) y React + TypeScript por features; cada tarjeta nombra su entregable en la capa correspondiente
Alcance planeado	31 tarjetas · 220 h	106 tarjetas · 745 h (672 h de desarrollo y 73 h de gestión), más el backlog posterior de 23 historias
Reparto entre integrantes	Desarrollo: 16 a 64 h por persona; gestión: 3 a 18 h	Desarrollo: 165 a 170 h por persona; gestión: 18 a 19 h por persona; cada integrante participa en servidor, interfaz y pruebas
Estado de las tarjetas	Con resultados ya registrados (minutas firmadas, aprobaciones)	Todas en POR HACER: las columnas EN ELABORACIÓN y REALIZADO se completan conforme se ejecute cada sprint

Tabla 1. Cambios respecto del tablero anterior.
Elaboración propia.

Capacidad. El tablero planea 745 horas, alrededor de 18.6 h por integrante por semana durante diez semanas. Es más de tres veces las 220 h del tablero anterior porque cubre la primera entrega completa. Si el equipo no dispone de esa capacidad, el recorte debe hacerse quitando historias completas, empezando por las de menor prioridad de la primera entrega, y no reduciendo actividades de cada historia; las historias son la unidad de entrega.

1. Marco metodológico

El equipo del proyecto SIGPI adopta el Proceso Unificado (RUP) como modelo de proceso de desarrollo de software y Scrum como metodología de seguimiento. Son complementarias: RUP aporta la disciplina de artefactos, arquitectura y trazabilidad, y Scrum la cadencia y la transparencia necesarias para gestionar un equipo reducido frente a un cliente institucional con disponibilidad limitada (contingencia de los riesgos RG-02 y RG-05).

Cada sprint de dos semanas se alinea con una fase del ciclo de vida de RUP. El Product Backlog se construye a partir de las historias de usuario, que derivan de los casos de uso del sistema y de la lista de dieciocho riesgos; cada historia se desglosa en actividades (tarjetas) con su entregable. La cadencia quincenal acordada con el Jefe del DTIC-FI es el Sprint Review institucional.

¿Cómo se da seguimiento al proyecto? El avance se mide en sprints de duración fija (dos semanas); al cierre de cada uno se actualiza el tablero de la sección 6, se revisa el incremento con el cliente y se registra la velocidad real (horas planeadas frente a ejecutadas) para replanificar. Este ciclo —planear, ejecutar, revisar, adaptar— se repite igual en los cinco sprints.

¿Cómo se identifica que se usa esta metodología? Tres señales la distinguen de un proceso en cascada o de un Scrum sin disciplina de artefactos: (1) artefactos formales de RUP —Visión, arquitectura, matriz de trazabilidad— entregados de forma incremental; (2) un tablero con cadencia fija cada dos semanas; y (3) la gestión de riesgos integrada al backlog, con una tarjeta de mitigación rastreable por cada riesgo.

Dimensión	Scrum (seguimiento)	RUP (modelo de proceso)
Unidad de avance	Sprint de 2 semanas	Iteración dentro de una fase
Artefactos clave	Product Backlog, Sprint Backlog, Incremento	Visión, historias, arquitectura, Risklog
Roles	PO · SM · Developers	Analista · Diseñador · Programador · Tester
Gestión de riesgos	Ítem del backlog con criterio de aceptación	Planificación dirigida por riesgos en cada iteración
Trazabilidad	No prescrita (delegada al equipo)	Obligatoria: historia → diseño → código → prueba
Comunicación con el cliente	Sprint Review quincenal	Hito de fase (Inicio, Elaboración, Construcción, Transición)

Tabla 2. Correspondencia entre las prácticas de Scrum y los artefactos y disciplinas de RUP adoptados por el equipo. Elaboración propia.

2. Justificación de la metodología elegida

La elección se justifica confrontando las características del proyecto contra los cinco factores críticos que Boehm y Turner (2003) proponen para decidir el balance entre métodos ágiles y dirigidos por plan: tamaño, criticidad, dinamismo, personal y cultura.

Factor	Característica del proyecto	Zona ágil o disciplinada (Boehm y Turner, 2003)	Práctica adoptada
Tamaño	Equipo de 4 integrantes; alcance acotado a un semestre (5 sprints).	Los equipos pequeños se ubican en la zona ágil: la coordinación cara a cara sustituye a la documentación extensa.	Scrum aporta la cadencia ligera de seguimiento (Sprint Backlog, Daily Scrum).
Criticidad	Sistema de soporte interno; no compromete vidas ni pérdidas catastróficas, pero exige trazabilidad auditable por evaluarse con rúbrica.	Una criticidad media no exige el rigor documental completo de un proceso dirigido por plan, pero no tolera la ausencia de artefactos formales.	RUP aporta la trazabilidad historia → código → prueba sin la sobrecarga de la cascada.
Dinamismo	El volumen de incidencias cambia y el clasificador no cuenta con datos históricos verificados (RT-01).	La alta incertidumbre de requisitos es el escenario que se asocia con la zona ágil.	Scrum permite replanificar el backlog cada sprint conforme se dispone de datos reales.
Personal	Cuatro estudiantes con experiencia en desarrollo pero sin rodaje como equipo Scrum.	Un equipo con experiencia técnica pero sin rodaje ágil se beneficia de plantillas y roles predefinidos.	Las plantillas de artefactos de RUP reducen la curva de aprendizaje.
Cultura	Cliente institucional con disponibilidad limitada a sesiones quincenales.	Una cultura de cliente distante favorece hitos de revisión espaciados y bien preparados.	La cadencia quincenal de Scrum se ajusta a la disponibilidad real del cliente y coincide con un hito de fase.

Tabla 3. Justificación metodológica frente a los cinco factores críticos de selección. Elaboración propia con base en Boehm y Turner (2003).

Conclusión. Ningún factor cae limpiamente en un extremo: el tamaño y el dinamismo empujan hacia la agilidad; la criticidad académica y la inexperiencia del equipo como unidad Scrum, hacia la disciplina de artefactos. Es la situación más

común en la práctica y la que motiva un método híbrido: Scrum gobierna el ritmo y la relación con el cliente; RUP gobierna qué se documenta y cómo se demuestra la trazabilidad de cada requisito hasta su prueba.

3. Arquitectura de referencia: React + TypeScript y FastAPI en capas

Las tarjetas nombran su entregable en la capa que le corresponde. La regla de dependencia es única: cada capa solo conoce a la inmediatamente inferior, y el dominio no depende de ninguna. Así, las reglas de negocio (por ejemplo, la matriz de prioridad RN-05) se prueban sin base de datos ni servidor web.

3.1 Backend: FastAPI con arquitectura en capas

Capa	Paquete	Responsabilidad
Presentación (API)	api/routers · schemas · dependencias	Recibe la petición HTTP, valida con esquemas Pydantic, invoca un servicio y devuelve la respuesta. No contiene reglas de negocio.
Aplicación (servicios)	services/	Orquesta los casos de uso (registrar, clasificar, asignar, escalar) y aplica las reglas RN-xx. Es la única capa que coordina repositorios e integraciones.
Dominio	domain/	Reglas puras y estados: matriz urgencia-impacto, puntaje de idoneidad, reloj del SLA. Sin dependencias de framework.
Acceso a datos	repositories/ · models/ · alembic/	Modelos SQLAlchemy y repositorios que encapsulan las consultas; las migraciones versionan el esquema.
Integración	integrations/	Adaptadores hacia el directorio institucional, el servicio de correo, el repositorio de inventario y el clasificador.
Transversal	core/	Configuración por entorno, seguridad (JWT y roles), excepciones, registro de eventos y auditoría.

Tabla 4. Capas del backend y su responsabilidad.
Elaboración propia.

3.2 Frontend: React + TypeScript por features

Elemento	Responsabilidad
app / routes	Enrutamiento (React Router), proveedores globales y protección de rutas por rol.
features/<dominio>	Un directorio por dominio (solicitudes, clasificacion, bandeja, sla, catalogos, usuarios) con sus páginas, componentes y hooks; es la unidad de trabajo de las tarjetas de interfaz.
api/	Cliente tipado; los tipos TypeScript se generan del esquema OpenAPI de FastAPI para que el contrato no se duplique a mano.
Estado de servidor	TanStack Query para consultas y mutaciones; TanStack Table para bandejas y listados.
shared/ui	Componentes reutilizables (shadcn/ui) y utilidades sin dependencia de un dominio.
Pruebas	Vitest con React Testing Library para componentes y hooks; TypeScript estricto y ESLint en el pipeline CI.

Tabla 5. Organización del frontend.
Elaboración propia.

3.3 Pruebas y calidad

El backend usa pytest con httpx y una base de datos de prueba; la cobertura mínima es del 60 % al cierre del sprint 4 y del 80 % al cierre del proyecto. El pipeline CI ejecuta ruff, pytest, el compilador de TypeScript, ESLint y Vitest en cada solicitud de cambio, y ninguna tarjeta de desarrollo se da por terminada sin su prueba asociada.

4. Leyenda y convenciones del tablero

HS — Haide Sánchez · LR — Laura Rodríguez · JI — Jesús Inieta · EB — Emmanuel Bernal. Cada tarjeta lleva un identificador (Sn-nn), su descripción, la categoría, el responsable, las horas, las historias de las que deriva y su entregable (→). Las tarjetas con borde azul son de **Desarrollo** (análisis, diseño, código, pruebas); las de borde ámbar son de **Gestión** (planeación, revisiones, comunicación con el cliente, control de riesgos).

Tipo	Horas	Alcance
D — Diseño	6 h	Modelos, contratos y reglas antes de codificar
B — Servidor	10 h	Routers, servicios, repositorios y esquemas
F — Interfaz	9 h	Páginas, componentes y hooks en React + TypeScript
I — Integración	8 h	Adaptadores hacia servicios institucionales y pasos que conectan capas
M — Datos y modelo	12 h	Conjunto de datos y entrenamiento del clasificador
P — Pruebas	7 h	Pruebas automatizadas de los criterios de aceptación
A — Análisis	6 h	Visión y contexto
X — Documentación	5 h	Despliegue y manual

Tabla 6. Horas planeadas por tipo de actividad de desarrollo.
Elaboración propia. Los valores se calibraron para que el promedio (8.3 h por actividad) coincidiera con el del Estudio de Factibilidad; son horas planeadas, no medidas.

5. Distribución de actividades por integrante

Todos los integrantes participan tanto en desarrollo como en gestión. La asignación se hizo balanceando las horas de cada categoría y las horas de servidor, interfaz y pruebas de cada persona; ningún integrante concentra un solo tipo de actividad.

Integrante	Gestión	Desarrollo	Total
Haide Sánchez	6 tareas · 18 h	20 tareas · 170 h	26 tareas · 188 h
Laura Rodríguez	6 tareas · 19 h	21 tareas · 165 h	27 tareas · 184 h
Jesús Iniesta	6 tareas · 18 h	21 tareas · 170 h	27 tareas · 188 h
Emmanuel Bernal	6 tareas · 18 h	20 tareas · 167 h	26 tareas · 185 h
Total	24 tareas · 73 h	82 tareas · 672 h	106 tareas · 745 h

Tabla 7. Distribución final de actividades por integrante, por categoría y carga horaria.
Elaboración propia.

Integrante	Servidor	Interfaz	Pruebas	Integración	Datos, diseño y otros
Haide Sánchez	50	27	35	16	42
Laura Rodríguez	60	27	35	8	35
Jesús Iniesta	60	36	35	16	23
Emmanuel Bernal	60	36	35	24	12

Tabla 8. Horas de desarrollo por integrante y por tipo de actividad.
Elaboración propia.

Integrante	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
Haide Sánchez	24	36	39	42	47
Laura Rodríguez	27	32	38	41	46
Jesús Iniesta	16	45	36	42	49
Emmanuel Bernal	16	41	36	50	42

Tabla 9. Horas por integrante y por sprint.
Elaboración propia.

La diferencia entre la persona con más y con menos horas es de 5 h en desarrollo (3 % del promedio) y de 1 h en gestión. Los roles de Scrum (Haide como Product Owner y Emmanuel como Scrum Master) no reducen su carga de programación: la gestión de ceremonias, riesgos y aceptación está repartida por tarjetas y no concentrada en los roles.

6. Tablero Kanban por sprint

Las tarjetas de cada sprint están en POR HACER porque el tablero es el plan; las columnas EN ELABORACIÓN y REALIZADO se completan durante la ejecución, con el límite de trabajo en curso que acuerde el equipo.

Sprint 1 · RUP: Inicio

14 tarjetas · 83 h (Desarrollo 67 h, Gestión 16 h) · ≈ 10.4 h por integrante por semana · Historias: HU-01.1, HU-02.3, HU-08.1, HU-10.1, HU-11.1

POR HACER (14)	EN ELABORACIÓN (0)	REALIZADO (0)
S1-01 · Redactar la Visión del Producto Documento RUP: problema, posicionamiento, interesados y características clave, alineado con el Modelado de Negocios (v2). Desarrollo (análisis) · Haide Sánchez · 6 h → Vision.md	Se actualiza durante el sprint	Se actualiza durante el sprint
S1-02 · Diagrama de contexto del sistema Actores humanos y de sistema (ACT-01 a ACT-10) y las interfaces de comunicación del SIGPI, en UML. Desarrollo (análisis) · Laura Rodríguez · 6 h → Diagrama de contexto		
S1-03 · Definir la arquitectura en capas (ADR) Registrar la decisión de capas del backend y del frontend, las reglas de dependencia entre capas y las convenciones de nombres y de pruebas. Desarrollo (diseño) · Jesús Iniesta · 6 h → docs/adr/0001-arquitectura-en-capas.md		
S1-04 · Estructura base de FastAPI en capas Proyecto con paquetes api/routers, schemas, services, domain, repositories, models, integrations y core; configuración por entorno. Desarrollo (servidor) · Emmanuel Bernal · 10 h → backend/app/ (esqueleto en capas)		
S1-05 · Estructura base de React + TypeScript Vite con TypeScript estricto, React Router, TanStack Query, shadcn/ui, ESLint y Prettier; carpetas api, features, shared y routes. Desarrollo (interfaz) · Haide Sánchez · 9 h → frontend/src/ (esqueleto por features)		
S1-06 · Definir roles y permisos por módulo Definir roles y permisos por módulo Desarrollo (diseño) · Laura Rodríguez · 6 h · HU-10.1 → core/permissions.py · docs/matriz_permisos.md		
S1-07 · Modelar acuerdos de nivel de servicio versionados Modelar los acuerdos versionados por categoría y prioridad Desarrollo (diseño) · Jesús Iniesta · 6 h · HU-08.1 → models/acuerdo_servicio.py · domain/sla.py		
S1-08 · Modelar áreas, especialidades, turnos y cargas máximas Modelar áreas, especialidades, turnos y cargas máximas Desarrollo (diseño) · Emmanuel Bernal · 6 h · HU-11.1 → models/tecnico.py · models/area.py		
S1-09 · Diseñar la entidad Solicitud y el contrato del alta Diseñar la entidad Solicitud (folio, estatus, ubicación) y el contrato del alta Desarrollo (diseño) · Laura Rodríguez · 6 h · HU-01.1 → models/solicitud.py · schemas/solicitud.py		
S1-10 · Matriz urgencia-impacto parametrizada Formalizar la matriz urgencia-impacto como tabla de datos Desarrollo (diseño) · Haide Sánchez · 6 h · HU-02.3 → domain/priority_matrix.py		
S1-11 · Acta de arranque y roles Scrum Asignar PO, SM y Developers, acordar la Definition of Ready y de Done y documentarlo en el acta de arranque. Gestión · Jesús Iniesta · 4 h → Acta de kick-off		
S1-12 · Product Backlog v1 desde las historias Cargar las 41 historias con prioridad MoSCoW, dependencias y actividades, y ordenar las de la primera entrega. Gestión · Laura Rodríguez · 6 h → Product Backlog v1		

POR HACER (14)	EN ELABORACIÓN (0)	REALIZADO (0)
S1-13 · Entrevista de validación con el Jefe del DTIC-FI Validar historias, prioridades y restricciones con el Jefe del Departamento. Mitigación de RR-01. Gestión · Haide Sánchez · 3 h → Minuta de reunión		
S1-14 · Plan de gestión de riesgos Asociar cada riesgo de la lista de dieciocho a una tarjeta de mitigación rastreable. Gestión · Laura Rodríguez · 3 h → RiskLog v1		

Cómo ejecutar el Sprint 1 (RUP: Inicio)

1. Kick-off: definir roles Scrum (PO → Haide, SM → Emmanuel, Developers → Laura y Jesús) y acordar la Definition of Ready y de Done; la gestión se reparte por igual entre los cuatro.
2. Cargar las 41 historias de usuario en el Product Backlog v1 con prioridad MoSCoW y dependencias; ordenar las de la primera entrega (18 historias «Debe»).
3. Validar historias y alcance con el Jefe del DTIC-FI (mitigación de RR-01) y producir la Visión y el diagrama de contexto.
4. Fijar la arquitectura en capas (ADR) y crear los esqueletos de FastAPI y de React + TypeScript que usará todo el equipo.
5. Diseñar el dominio de las historias cimiento (HU-10.1, HU-08.1, HU-11.1, HU-01.1 y HU-02.3). Sprint Review: presentar backlog, visión y arquitectura. Hito de cierre de Inicio.

Sprint 2 · RUP: Elaboración

21 tarjetas · 154 h (Desarrollo 144 h, Gestión 10 h) · ≈ 19.2 h por integrante por semana · Historias: HU-01.1, HU-04.1, HU-08.1, HU-10.1, HU-10.3, HU-11.1

POR HACER (21)	EN ELABORACIÓN (0)	REALIZADO (0)
S2-01 · Modelo relacional base y migraciones Modelos SQLAlchemy y migraciones Alembic de las entidades del núcleo; datos semilla de categorías y prioridades. Desarrollo (servidor) · Jesús Iniesta · 10 h → models/ · alembic/versions/	Se actualiza durante el sprint	Se actualiza durante el sprint
S2-02 · Pipeline CI y Docker Compose GitHub Actions con ruff, pytest, tsc, ESLint y Vitest; contenedores de API, base de datos y frontend. Desarrollo (integración) · Emmanuel Bernal · 8 h → .github/workflows/ci.yml · docker-compose.yml		
S2-03 · Cliente API tipado desde OpenAPI Generar los tipos TypeScript a partir del esquema OpenAPI de FastAPI y encapsular las llamadas en api/. Desarrollo (interfaz) · Laura Rodríguez · 9 h → frontend/src/api/ (tipos generados)		
S2-04 · Servicio de cuentas y roles sincronizado con el directorio Implementar alta, modificación y baja de cuentas sincronizadas con el directorio Desarrollo (servidor) · Haide Sánchez · 10 h · HU-10.1 → api/routers/usuarios.py · services/usuario_service.py · repositories/usuario_repository.py		
S2-05 · Pantalla de administración de usuarios Construir la administración de usuarios Desarrollo (interfaz) · Jesús Iniesta · 9 h · HU-10.1 → features/usuarios/pages/UsuariosPage.tsx · api/usuarios.ts		
S2-06 · Pruebas de alta de cuentas y cuenta no reconocida Probar el alta válida y la cuenta no reconocida Desarrollo (pruebas) · Emmanuel Bernal · 7 h · HU-10.1 → tests/api/test_usuarios.py · UsuariosPage.test.tsx		
S2-07 · Modelo de auditoría inalterable Definir el registro de auditoría inalterable Desarrollo (diseño) · Laura Rodríguez · 6 h · HU-10.3 → models/auditoria.py · migración Alembic		
S2-08 · Servicio de auditoría de cambios Registrar autor, fecha y valores anterior y nuevo en cada cambio Desarrollo (servidor) · Laura Rodríguez · 10 h · HU-10.3 → services/auditoria_service.py · repositories/auditoria_repository.py		

POR HACER (21)	EN ELABORACIÓN (0)	REALIZADO (0)
S2-09 • Pruebas de inalterabilidad de la auditoría Probar que el registro no puede modificarse Desarrollo (pruebas) · Haide Sánchez · 7 h · HU-10.3 → tests/services/test_auditoria.py		
S2-10 • Servicio del catálogo de técnicos y áreas Implementar el mantenimiento del catálogo con vigencia inmediata Desarrollo (servidor) · Emmanuel Bernal · 10 h · HU-11.1 → api/routers/catalogos.py · services/catalogo_service.py · repositories/tecnico_repository.py		
S2-11 • Pantalla del catálogo de técnicos y áreas Construir la pantalla de administración del catálogo Desarrollo (interfaz) · Jesús Iniesta · 9 h · HU-11.1 → features/catalogos/pages/CatalogoTecnicosPage.tsx		
S2-12 • Pruebas de vigencia inmediata del catálogo Probar que el motor usa los datos vigentes Desarrollo (pruebas) · Jesús Iniesta · 7 h · HU-11.1 → tests/api/test_catalogos.py		
S2-13 • Servicio de versiones de SLA Implementar las versiones, el histórico y el acuerdo predeterminado Desarrollo (servidor) · Haide Sánchez · 10 h · HU-08.1 → api/routers/sla.py · services/sla_service.py · repositories/sla_repository.py		
S2-14 • Pantalla de configuración de SLA Construir la pantalla de configuración con validaciones Desarrollo (interfaz) · Emmanuel Bernal · 9 h · HU-08.1 → features/sla/pages/SlaConfigPage.tsx · features/sla/hooks/useSla.ts		
S2-15 • Pruebas de versiones e inconsistencias del SLA Probar versión nueva, inconsistencia y combinación sin acuerdo Desarrollo (pruebas) · Laura Rodríguez · 7 h · HU-08.1 → tests/services/test_sla_service.py		
S2-16 • Servicio de alta con folio y validaciones RN-02 y RN-03 Implementar el alta con folio consecutivo y las validaciones de RN-02 y RN-03 Desarrollo (servidor) · Jesús Iniesta · 10 h · HU-01.1 → api/routers/solicitudes.py · services/solicitud_service.py · repositories/solicitud_repository.py		
S2-17 • Modelo de la bitácora de atención Definir la bitácora: marca de tiempo, autor, estatus anterior y nuevo Desarrollo (diseño) · Haide Sánchez · 6 h · HU-04.1 → models/bitacora.py · schemas/bitacora.py		
S2-18 • Planeación del Sprint 2 y refinamiento Estimar y confirmar las historias del sprint con el Product Owner. Gestión · Emmanuel Bernal · 3 h → Sprint Backlog 2		
S2-19 • Revisión del catálogo de riesgos Actualizar probabilidad e impacto con la información del sprint 1 (RT-01 y RG-02). Gestión · Haide Sánchez · 3 h → RiskLog v2		
S2-20 • Sprint Review 2 con el DTIC-FI Presentar el incremento y registrar la retroalimentación. Gestión · Emmanuel Bernal · 2 h → Minuta SR-2		
S2-21 • Retrospectiva 2 y velocidad Comparar horas planeadas y ejecutadas y ajustar el siguiente sprint. Gestión · Emmanuel Bernal · 2 h → Registro de velocidad		

Cómo ejecutar el Sprint 2 (RUP: Elaboración)

1. Construir la base ejecutable: modelos y migraciones, pipeline CI, Docker Compose y cliente API tipado desde OpenAPI.
2. Entregar las historias cimienta en todas sus capas: cuentas y roles (HU-10.1), auditoría (HU-10.3), catálogo de técnicos (HU-11.1) y acuerdos de servicio (HU-08.1).
3. Implementar el servicio de alta de solicitudes (HU-01.1) y diseñar la bitácora de atención (HU-04.1).
4. Mitigar RT-01 desde ya: decidir el corte de reglas explícitas para la clasificación antes de tener datos.
5. Sprint Review + Retrospectiva: registrar velocidad real frente a planeada y ajustar el backlog.

Sprint 3 · RUP: Construcción I

21 tarjetas · 149 h (Desarrollo 137 h, Gestión 12 h) · ≈ 18.6 h por integrante por semana · Historias: HU-01.1, HU-01.3, HU-02.3, HU-04.1, HU-04.2

POR HACER (21)	EN ELABORACIÓN (0)	REALIZADO (0)
S3-01 · Manejo transversal de errores y registro Excepciones de dominio, manejador global en FastAPI, formato uniforme de error y registro estructurado de eventos. Desarrollo (servidor) · Laura Rodríguez · 10 h → core/exceptions.py · core/logging.py	Se actualiza durante el sprint	Se actualiza durante el sprint
S3-02 · Formulario de registro de solicitud Construir el formulario de captura con validación en línea y acuse Desarrollo (interfaz) · Emmanuel Bernal · 9 h · HU-01.1 → features/solicitudes/components/SolicitudForm.tsx · hooks/useCrearSolicitud.ts		
S3-03 · Autenticación con el directorio institucional Integrar la autenticación con el directorio institucional Desarrollo (integración) · Haide Sánchez · 8 h · HU-01.1 → integrations/directorio_client.py · core/security.py		
S3-04 · Pruebas del alta de solicitud Automatizar los tres criterios: alta válida, campos incompletos y credenciales inválidas Desarrollo (pruebas) · Jesús Iniesta · 7 h · HU-01.1 → tests/api/test_solicitudes.py · SolicitudForm.test.tsx		
S3-05 · Servicio de cálculo de prioridad Calcular la prioridad de forma centralizada y bloquear su edición directa Desarrollo (servidor) · Emmanuel Bernal · 10 h · HU-02.3 → services/prioridad_service.py		
S3-06 · Pruebas de las nueve combinaciones de la matriz Probar las nueve combinaciones de la matriz y el intento de edición directa Desarrollo (pruebas) · Laura Rodríguez · 7 h · HU-02.3 → tests/domain/test_priority_matrix.py		
S3-07 · Endpoint de registro asistido Habilitar el registro asistido con canal de origen (teléfono o presencial) Desarrollo (servidor) · Haide Sánchez · 10 h · HU-01.3 → api/routers/solicitudes.py (POST /asistido) · services/solicitud_service.py		
S3-08 · Pantalla de registro asistido Agregar la variante de captura para el técnico con búsqueda del solicitante Desarrollo (interfaz) · Laura Rodríguez · 9 h · HU-01.3 → features/solicitudes/pages/RegistroAsistidoPage.tsx		
S3-09 · Contingencia cuando el directorio no responde Definir la contingencia cuando el directorio no responde (marca «por verificar») Desarrollo (integración) · Jesús Iniesta · 8 h · HU-01.3 → integrations/directorio_client.py (respaldo) · domain/estados.py		
S3-10 · Pruebas de registro asistido Probar el registro asistido y la contingencia Desarrollo (pruebas) · Emmanuel Bernal · 7 h · HU-01.3 → tests/api/test_registro_asistido.py		
S3-11 · Servicio de seguimiento y resolución Registrar diagnóstico, solución y cambios de estatus; calcular el tiempo de resolución Desarrollo (servidor) · Haide Sánchez · 10 h · HU-04.1 → api/routers/seguimiento.py · services/seguimiento_service.py		
S3-12 · Detalle de la solicitud con su bitácora Construir el detalle de la solicitud con su bitácora Desarrollo (interfaz) · Jesús Iniesta · 9 h · HU-04.1 → features/solicitudes/pages/SolicitudDetallePage.tsx		
S3-13 · Pruebas de resolución y confirmación Probar la resolución y la solicitud de confirmación Desarrollo (pruebas) · Laura Rodríguez · 7 h · HU-04.1 → tests/services/test_seguimiento.py		
S3-14 · Estado «En espera» y reloj del SLA Implementar el estado «En espera» y la suspensión y reanudación del reloj del SLA Desarrollo (servidor) · Emmanuel Bernal · 10 h · HU-04.2 → services/sla_clock_service.py · domain/estados.py		
S3-15 · Motivo de espera en la pantalla de atención Agregar el motivo de espera en la pantalla de atención Desarrollo (interfaz) · Haide Sánchez · 9 h · HU-04.2 → features/solicitudes/components/EsperaDialog.tsx		

POR HACER (21)	EN ELABORACIÓN (0)	REALIZADO (0)
S3-16 · Pruebas de suspensión y reanudación del reloj Probar el descuento del periodo suspendido Desarrollo (pruebas) · Jesús Iniesta · 7 h · HU-04.2 → tests/services/test_sla_clock.py		
S3-17 · Planeación del Sprint 3 Confirmar el alcance del sprint y reservar el 20 % para contingencias. Gestión · Jesús Iniesta · 3 h → Sprint Backlog 3		
S3-18 · Estrategia de ramas y revisión de pull requests Definir el flujo de ramas, las reglas de revisión de código y el criterio de fusión. Gestión · Laura Rodríguez · 3 h → Guía de contribución		
S3-19 · Revisión de riesgos del sprint 3 Actualizar el RiskLog con la información del incremento. Gestión · Haide Sánchez · 2 h → RiskLog v3		
S3-20 · Sprint Review 3 con el DTIC-FI Presentar el flujo de alta y seguimiento y registrar nuevos requisitos. Gestión · Jesús Iniesta · 2 h → Minuta SR-3		
S3-21 · Retrospectiva 3 y velocidad Evaluar velocidad real frente a estimada y replanificar. Gestión · Laura Rodríguez · 2 h → Registro de velocidad		

Cómo ejecutar el Sprint 3 (RUP: Construcción I)

1. Primer incremento funcional: el flujo de alta (HU-01.1), el registro asistido (HU-01.3) y el seguimiento con bitácora (HU-04.1) operan de punta a punta.
2. Calcular la prioridad con la matriz urgencia-impacto (HU-02.3) y suspender el reloj del SLA (HU-04.2).
3. Reservar el 20 % del tiempo del sprint para contingencias de RT-01 y RR-03; aplicar el flujo de ramas y revisión de pull requests.
4. Sprint Review con el Jefe del DTIC-FI (contingencia RG-05) y registrar la retroalimentación.
5. Retrospectiva: evaluar velocidad real frente a estimada y replanificar el sprint 4.

Sprint 4 · RUP: Construcción II

24 tarjetas · 175 h (Desarrollo 161 h, Gestión 14 h) · ≈ 21.9 h por integrante por semana · Historias: HU-02.1, HU-02.2, HU-04.3, HU-07.1, HU-07.2

POR HACER (24)	EN ELABORACIÓN (0)	REALIZADO (0)
S4-01 · Base de pruebas de integración y de componentes Fixtures de pytest con base de datos de prueba y configuración de Vitest con React Testing Library. Desarrollo (pruebas) · Laura Rodríguez · 7 h → tests/conftest.py · vite.config.ts	Se actualiza durante el sprint	Se actualiza durante el sprint
S4-02 · Consulta por folio y confirmación de la solución Exponer la consulta por folio y la confirmación de la solución Desarrollo (servidor) · Jesús Iniesta · 10 h · HU-04.3 → api/routers/solicitudes.py (GET /{folio}, POST /confirmar)		
S4-03 · Vista de estatus para el usuario Construir la vista de estatus para el usuario Desarrollo (interfaz) · Emmanuel Bernal · 9 h · HU-04.3 → features/solicitudes/pages/MiSolicitudPage.tsx		
S4-04 · Pruebas de consulta y cierre por confirmación Probar la consulta y el cierre por confirmación Desarrollo (pruebas) · Haide Sánchez · 7 h · HU-04.3 → tests/api/test_consulta_folio.py		
S4-05 · Catálogo de eventos y plantillas de aviso Definir el catálogo de eventos, destinatarios y plantillas de aviso Desarrollo (diseño) · Laura Rodríguez · 6 h · HU-07.1 → domain/eventos.py · templates/avisos/		

POR HACER (24)	EN ELABORACIÓN (0)	REALIZADO (0)
S4-06 · Servicio de notificaciones por evento Emitir avisos por evento con folio, prioridad, técnico y fecha compromiso Desarrollo (servidor) · Laura Rodríguez · 10 h · HU-07.1 → services/notificacion_service.py · repositories/notificacion_repository.py		
S4-07 · Integración con el servicio de correo Integrar el servicio de correo y registrar el historial Desarrollo (integración) · Emmanuel Bernal · 8 h · HU-07.1 → integrations/correo_client.py		
S4-08 · Pruebas de entrega de avisos Probar la entrega en menos de un minuto Desarrollo (pruebas) · Haide Sánchez · 7 h · HU-07.1 → tests/services/test_notificacion.py		
S4-09 · Monitor del 80 % del tiempo comprometido Detectar el 80 % del tiempo de resolución comprometido Desarrollo (servidor) · Jesús Iniesta · 10 h · HU-07.2 → services/sla_monitor_service.py · tarefas programadas		
S4-10 · Alertas al técnico y al Jefe Emitir la alerta al técnico y al Jefe, en la aplicación y por correo Desarrollo (integración) · Laura Rodríguez · 8 h · HU-07.2 → services/notificacion_service.py (alerta_riesgo)		
S4-11 · Pruebas del umbral de alerta Probar la desviación máxima de un minuto Desarrollo (pruebas) · Emmanuel Bernal · 7 h · HU-07.2 → tests/services/test_sla_monitor.py		
S4-12 · Conjunto de datos etiquetado Construir y etiquetar el conjunto de entrenamiento con vocabulario del DTIC Desarrollo (datos y modelo) · Haide Sánchez · 12 h · HU-02.1 → ml/etl/build_dataset.py · ml/data/dataset_v1.csv		
S4-13 · Servicio de clasificación (reglas y modelo) Exponer el clasificador como servicio y guardar versión y confianza Desarrollo (servidor) · Jesús Iniesta · 10 h · HU-02.1 → services/clasificacion_service.py · integrations/clasificador_client.py		
S4-14 · Integración de la clasificación en el alta Invocar el clasificador al registrar y pasar el estatus a «Clasificada» Desarrollo (integración) · Emmanuel Bernal · 8 h · HU-02.1 → services/solicitud_service.py (paso de clasificación)		
S4-15 · Umbral de confianza configurable Definir el umbral de confianza configurable (0.75 por defecto) Desarrollo (diseño) · Emmanuel Bernal · 6 h · HU-02.2 → core/config.py · domain/umbral.py		
S4-16 · Encolamiento de predicciones inciertas Enviar a la cola de revisión las predicciones bajo el umbral Desarrollo (servidor) · Laura Rodríguez · 10 h · HU-02.2 → services/revision_service.py · repositories/revision_repository.py		
S4-17 · Pantalla de revisión de clasificaciones Construir la pantalla de revisión con edición y justificación obligatoria Desarrollo (interfaz) · Haide Sánchez · 9 h · HU-02.2 → features/clasificacion/pages/RevisionPage.tsx		
S4-18 · Registro de correcciones como ejemplos Guardar cada corrección como ejemplo para el reentrenamiento Desarrollo (servidor) · Jesús Iniesta · 10 h · HU-02.2 → repositories/correccion_repository.py		
S4-19 · Pruebas de revisión y corrección Probar predicción incierta y corrección con justificación Desarrollo (pruebas) · Emmanuel Bernal · 7 h · HU-02.2 → tests/api/test_revision.py		
S4-20 · Planeación del Sprint 4 Confirmar el alcance y ordenar las dependencias de notificaciones y clasificación. Gestión · Emmanuel Bernal · 3 h → Sprint Backlog 4		
S4-21 · Matriz de trazabilidad historia → código → prueba Vincular cada historia con su módulo, su prueba y su criterio de aceptación. Gestión · Haide Sánchez · 5 h → Trazabilidad.xlsx		

POR HACER (24)	EN ELABORACIÓN (0)	REALIZADO (0)
S4-22 · Revisión de riesgos del sprint 4 Revisar la exactitud del clasificador y la disponibilidad de datos (RT-01). Gestión · Emmanuel Bernal · 2 h → RiskLog v4		
S4-23 · Sprint Review 4 con el DTIC-FI Demostrar notificaciones y clasificación con revisión humana. Gestión · Haide Sánchez · 2 h → Minuta SR-4		
S4-24 · Retrospectiva 4 y velocidad Evaluar la velocidad y ajustar el último sprint. Gestión · Jesús Iniesta · 2 h → Registro de velocidad		

Cómo ejecutar el Sprint 4 (RUP: Construcción II)

1. Segundo incremento: consulta de estatus para el usuario (HU-04.3), notificaciones (HU-07.1) y alertas de riesgo de incumplimiento (HU-07.2).
2. Clasificación con revisión humana: conjunto de datos, servicio de clasificación e integración (HU-02.1), y umbral y cola de revisión (HU-02.2).
3. Iniciar la recolección de datos etiquetados para entrenar el modelo y mitigar RT-01; el clasificador arranca con reglas explícitas.
4. Construir la matriz de trazabilidad historia → código → prueba y la base de pruebas de integración y de componentes.
5. Sprint Review: demostrar notificaciones y clasificación con revisión humana.

Sprint 5 · RUP: Transición

26 tarjetas · 184 h (Desarrollo 163 h, Gestión 21 h) · ≈ 23.0 h por integrante por semana · Historias: HU-02.1, HU-02.5, HU-03.1, HU-03.3, HU-04.6

POR HACER (26)	EN ELABORACIÓN (0)	REALIZADO (0)
S5-01 · Ampliar la cobertura de pruebas al 80 % Completar pruebas de servicios y de dominio hasta el umbral de cobertura mínimo para el cierre. Desarrollo (pruebas) · Haide Sánchez · 7 h → Informe de cobertura	Se actualiza durante el sprint	Se actualiza durante el sprint
S5-02 · Documentación de despliegue README con requisitos, variables de entorno, Docker Compose, respaldo de base de datos y reversión. Desarrollo (documentación) · Laura Rodríguez · 5 h → DEPLOY.md		
S5-03 · Manual de usuario Guía para el usuario solicitante, el técnico, el Jefe del Departamento y el administrador. Desarrollo (documentación) · Jesús Iniesta · 5 h → ManualUsuario.pdf		
S5-04 · Consulta de la bandeja del técnico Consultar las solicitudes asignadas con orden por prioridad y tiempo restante Desarrollo (servidor) · Laura Rodríguez · 10 h · HU-04.6 → api/routers/bandeja.py · repositories/solicitud_repository.py (filtros)		
S5-05 · Tabla de bandeja con filtros y paginación Construir la tabla paginada con filtros por estatus, prioridad y técnico Desarrollo (interfaz) · Emmanuel Bernal · 9 h · HU-04.6 → features/bandeja/pages/BandejaPage.tsx (TanStack Table)		
S5-06 · Pruebas de orden, filtros y paginación Probar orden, filtros y paginación con volumen alto Desarrollo (pruebas) · Jesús Iniesta · 7 h · HU-04.6 → tests/api/test_bandeja.py		
S5-07 · Entrenamiento y evaluación del clasificador Entrenar y evaluar el clasificador de categoría, urgencia e impacto Desarrollo (datos y modelo) · Haide Sánchez · 12 h · HU-02.1 → ml/training/train_classifier.py · ml/models/clasificador_v1.joblib		
S5-08 · Pruebas de desempeño de la clasificación Medir el tiempo de respuesta (menos de cinco segundos) y probar los criterios Desarrollo (pruebas) · Laura Rodríguez · 7 h · HU-02.1 → tests/services/test_clasificacion.py		

POR HACER (26)	EN ELABORACIÓN (0)	REALIZADO (0)
S5-09 · Consulta de la cola de revisión Consultar las solicitudes pendientes de revisión por antigüedad y prioridad Desarrollo (servidor) · Emmanuel Bernal · 10 h · HU-02.5 → api/routers/revision.py (GET /cola)		
S5-10 · Vista de la cola de revisión Construir la vista de la cola de revisión Desarrollo (interfaz) · Jesús Iniеста · 9 h · HU-02.5 → features/clasificacion/pages/ColaRevisionPage.tsx		
S5-11 · Pruebas de orden y estado vacío Probar orden y estado vacío Desarrollo (pruebas) · Haide Sánchez · 7 h · HU-02.5 → tests/api/testColaRevision.py		
S5-12 · Fórmula del puntaje de idoneidad Definir la fórmula del puntaje de idoneidad: especialidad, disponibilidad y carga Desarrollo (diseño) · Jesús Iniеста · 6 h · HU-03.1 → domain/assignment_scoring.py		
S5-13 · Servicio de asignación automática Seleccionar al candidato de mayor puntaje entre los técnicos elegibles Desarrollo (servidor) · Laura Rodríguez · 10 h · HU-03.1 → api/routers/asignacion.py · services/asignacion_service.py		
S5-14 · Cambio de estatus y aviso al técnico Pasar el estatus a «Asignada» y notificar al técnico Desarrollo (integración) · Haide Sánchez · 8 h · HU-03.1 → services/asignacion_service.py → notificacion_service		
S5-15 · Desglose del puntaje en la interfaz Mostrar el desglose del puntaje al técnico asignado Desarrollo (interfaz) · Laura Rodríguez · 9 h · HU-03.1 → features/solicitudes/components/PuntajeDesglose.tsx		
S5-16 · Pruebas de asignación por carga y especialidad Probar la asignación con distintas cargas y especialidades Desarrollo (pruebas) · Emmanuel Bernal · 7 h · HU-03.1 → tests/domain/test_assignment_scoring.py		
S5-17 · Regla de carga máxima y prioridad P1 Ignorar la carga máxima en P1 y encolar en la cola general las demás Desarrollo (servidor) · Haide Sánchez · 10 h · HU-03.3 → domain/assignment_rules.py		
S5-18 · Temporizador de cinco minutos para P1 Programar la alerta a los cinco minutos sin asignar una solicitud P1 Desarrollo (servidor) · Emmanuel Bernal · 10 h · HU-03.3 → services/asignacion_service.py (planificador)		
S5-19 · Alerta directa al Jefe del Departamento Enviar la alerta directa al Jefe del Departamento Desarrollo (integración) · Jesús Iniеста · 8 h · HU-03.3 → services/notificacion_service.py		
S5-20 · Pruebas de los tres escenarios de carga Probar los tres escenarios de carga Desarrollo (pruebas) · Jesús Iniеста · 7 h · HU-03.3 → tests/services/test_asignacion_p1.py		
S5-21 · Planeación del Sprint 5 y checklist de liberación Confirmar el alcance final y preparar la lista de verificación de despliegue. Gestión · Haide Sánchez · 3 h → Sprint Backlog 5		
S5-22 · Pruebas de aceptación con el DTIC-FI Ejecutar los escenarios acordados con el Jefe del Departamento y registrar defectos. Gestión · Emmanuel Bernal · 6 h → Acta de aceptación		
S5-23 · Sprint Review final y cierre formal Sesión de validación y aceptación del cliente (mitigación de RG-02). Gestión · Jesús Iniеста · 3 h → Minuta SR-5		
S5-24 · Presentación y cierre académico Demostración en vivo, lecciones aprendidas y mapa de cobertura de historias. Gestión · Jesús Iniеста · 4 h → Presentación final		

POR HACER (26)	EN ELABORACIÓN (0)	REALIZADO (0)
S5-25 · Retrospectiva del proyecto y métricas Consolidar velocidad, cobertura de historias y lecciones aprendidas. Gestión · Laura Rodríguez · 3 h → Informe de cierre		
S5-26 · Actualizar el estado del backlog posterior Documentar qué historias de las entregas 2 y 3 quedan pendientes y su prioridad. Gestión · Laura Rodríguez · 2 h → Product Backlog v5		

Cómo ejecutar el Sprint 5 (RUP: Transición)

1. Cerrar la primera entrega: bandeja del técnico (HU-04.6), cola de revisión (HU-02.5), asignación automática (HU-03.1) y regla de solicitudes críticas (HU-03.3).
2. Entrenar y evaluar el modelo (HU-02.1); si los datos son insuficientes, documentar el plan y mantener las reglas.
3. Ampliar la cobertura de pruebas al 80 %, producir la documentación de despliegue y el manual de usuario.
4. Pruebas de aceptación y Sprint Review final: sesión formal de validación con el DTIC-FI (mitigación de RG-02).
5. Retrospectiva del proyecto y actualización del backlog posterior con las historias de las entregas 2 y 3.

7. Resumen de actividades por sprint

Sprint	Fase RUP	Desarrollo	Gestión	Total tarjetas	Horas
1	RUP: Inicio	10	4	14	83
2	RUP: Elaboración	17	4	21	154
3	RUP: Construcción I	16	5	21	149
4	RUP: Construcción II	19	5	24	175
5	RUP: Transición	20	6	26	184
Total		82	24	106	745

Tabla 10. Resumen de actividades de Desarrollo y de Gestión por sprint.
Elaboración propia.

8. Trazabilidad: historias de usuario y tarjetas

Cada historia se vincula con las tarjetas que la implementan. Las de prioridad «Debe» están programadas en los sprints; las demás pasan al backlog posterior (sección 9).

Historia	Título	Actor	Prioridad	Sprint	Tarjetas
HU-01.1	Registrar una solicitud desde el portal	ACT-01	Debe	1, 2, 3	S1-09, S2-16, S3-02, S3-03, S3-04
HU-01.2	Indicar el equipo afectado y adjuntar evidencia	ACT-01	Debería	Posterior	—
HU-01.3	Registrar una solicitud recibida por teléfono o en persona	ACT-02	Debe	3	S3-07, S3-08, S3-09, S3-10
HU-01.4	Evitar solicitudes duplicadas	ACT-01	Debería	Posterior	—
HU-01.5	Registrar una solicitud correctiva durante un mantenimiento	ACT-03	Debería	Posterior	—
HU-02.1	Proponer categoría, urgencia e impacto automáticamente	ACT-02	Debe	4, 5	S4-12, S5-07, S4-13, S4-14, S5-08
HU-02.2	Revisar y corregir clasificaciones de baja confianza	ACT-02	Debe	4	S4-15, S4-16, S4-17, S4-18, S4-19
HU-02.3	Calcular la prioridad con la matriz urgencia-impacto	ACT-04	Debe	1, 3	S1-10, S3-05, S3-06
HU-02.4	Continuar el servicio si el motor no responde	ACT-02	Debería	Posterior	—
HU-02.5	Consultar la cola de revisión de clasificaciones	ACT-02	Debe	5	S5-09, S5-10, S5-11
HU-03.1	Proponer al técnico idóneo	ACT-04	Debe	5	S5-12, S5-13, S5-14, S5-15, S5-16

Historia	Título	Actor	Prioridad	Sprint	Tarjetas
HU-03.2	Sustituir manualmente al técnico propuesto	ACT-02	Debería	Posterior	—
HU-03.3	Garantizar la asignación de solicitudes críticas	ACT-04	Debe	5	S5-17, S5-18, S5-19, S5-20
HU-04.1	Registrar el avance en una bitácora única	ACT-02 · ACT-03	Debe	2, 3	S2-17, S3-11, S3-12, S3-13
HU-04.2	Suspender el reloj del SLA por causas ajenas	ACT-02 · ACT-03	Debe	3	S3-14, S3-15, S3-16
HU-04.3	Consultar el estatus y confirmar la solución	ACT-01	Debe	4	S4-02, S4-03, S4-04
HU-04.4	Reabrir una solicitud dentro del plazo	ACT-01	Debería	Posterior	—
HU-04.5	Cerrar automáticamente las solicitudes sin respuesta	ACT-04	Debería	Posterior	—
HU-04.6	Consultar la bandeja de solicitudes asignadas	ACT-02 · ACT-03	Debe	5	S5-04, S5-05, S5-06
HU-05.1	Solicitar un escalamiento con evidencia	ACT-03	Debería	Posterior	—
HU-05.2	Autorizar o rechazar el escalamiento	ACT-04	Debería	Posterior	—
HU-05.3	Escalar automáticamente solicitudes críticas sin respuesta	ACT-04	Podría	Posterior	—
HU-05.4	Recibir la solicitud escalada con su historial	ACT-03	Debería	Posterior	—
HU-06.1	Buscar soluciones antes de diagnosticar	ACT-02 · ACT-03	Debería	Posterior	—
HU-06.2	Documentar una solución nueva	ACT-03	Debería	Posterior	—
HU-06.3	Consultar artículos de autoayuda	ACT-01	Podría	Posterior	—
HU-07.1	Recibir avisos del ciclo de vida	ACT-01	Debe	4	S4-05, S4-06, S4-07, S4-08
HU-07.2	Alertar el riesgo de incumplimiento	ACT-02 · ACT-03	Debe	4	S4-09, S4-10, S4-11
HU-07.3	No perder avisos si falla el correo	ACT-01	Debería	Posterior	—
HU-08.1	Definir y versionar los acuerdos de nivel de servicio	ACT-04	Debe	1, 2	S1-07, S2-13, S2-14, S2-15
HU-09.1	Ver los indicadores del servicio	ACT-04	Debería	Posterior	—
HU-09.2	Exportar los indicadores	ACT-04	Debería	Posterior	—
HU-10.1	Administrar cuentas y roles	ACT-09	Debe	1, 2	S1-06, S2-04, S2-05, S2-06
HU-10.2	Proteger la continuidad de la administración	ACT-09	Debería	Posterior	—
HU-10.3	Auditar las acciones de administración	ACT-09	Debe	2	S2-07, S2-08, S2-09
HU-11.1	Mantener el catálogo de técnicos y áreas	ACT-09	Debe	1, 2	S1-08, S2-10, S2-11, S2-12
HU-11.2	Evitar categorías sin cobertura	ACT-09	Debería	Posterior	—
HU-11.3	Mantener el inventario de activos y el calendario de mantenimiento	ACT-04	Debería	Posterior	—
HU-11.4	Actualizar la ficha del activo y emitir el acta de mantenimiento	ACT-03	Debería	Posterior	—

HU-12.1	Supervisar la exactitud del clasificador	ACT-09	Podría	Posterior	—
HU-12.2	Reentrenar el modelo con las solicitudes verificadas	ACT-09	Podría	Posterior	—

Tabla 11. Matriz de trazabilidad entre las historias de usuario y las tarjetas del tablero.
Elaboración propia con base en las Historias de usuario, versión 3.

9. Backlog posterior (entregas 2 y 3)

Las 23 historias de prioridad «Debería» y «Podría» conservan sus 76 actividades, sin responsable ni sprint asignado; se planean en cuanto se cierre la primera entrega. Para asignarlas se aplicará el mismo criterio de equidad entre desarrollo y gestión.

Actividad	Entrega	Tipo	Descripción
HU-01.2-A1	Entrega 2	Diseño	Definir el modelo de adjuntos y su relación con el activo
HU-01.2-A2	Entrega 2	Servidor	Implementar la carga de archivos con límites de tamaño y tipo configurables
HU-01.2-A3	Entrega 2	Interfaz	Agregar el selector de activo (inventario o ubicación) y el campo de evidencia
HU-01.2-A4	Entrega 2	Pruebas	Probar activo no identificado y archivo fuera de límites
HU-01.4-A1	Entrega 2	Diseño	Especificar la regla de equivalencia: mismo activo y categoría en dos horas
HU-01.4-A2	Entrega 2	Servidor	Implementar la búsqueda de solicitudes equivalentes y la vinculación como relacionada
HU-01.4-A3	Entrega 2	Interfaz	Mostrar al usuario el folio de la solicitud existente
HU-01.4-A4	Entrega 2	Pruebas	Probar los límites de la ventana de dos horas
HU-01.5-A1	Entrega 2	Diseño	Definir el origen «mantenimiento preventivo» y el vínculo con la orden de trabajo
HU-01.5-A2	Entrega 2	Servidor	Crear la solicitud correctiva con el activo precargado
HU-01.5-A3	Entrega 2	Interfaz	Agregar la acción «Registrar anomalía» en la pantalla de mantenimiento
HU-01.5-A4	Entrega 2	Pruebas	Probar que la solicitud entra al flujo normal de clasificación
HU-02.4-A1	Entrega 2	Servidor	Definir el tiempo límite y el estado «pendiente de clasificación manual»
HU-02.4-A2	Entrega 2	Integración	Notificar al administrador cuando el motor no responde
HU-02.4-A3	Entrega 2	Pruebas	Simular la caída del motor
HU-03.2-A1	Entrega 2	Servidor	Permitir la sustitución con justificación y conservar la evidencia
HU-03.2-A2	Entrega 2	Interfaz	Agregar el selector de técnico alterno
HU-03.2-A3	Entrega 2	Pruebas	Probar la notificación a ambos técnicos
HU-04.4-A1	Entrega 2	Servidor	Implementar la reapertura con validación del plazo de cinco días hábiles
HU-04.4-A2	Entrega 2	Interfaz	Agregar la acción «Reabrir» y el mensaje de plazo vencido
HU-04.4-A3	Entrega 2	Pruebas	Probar el día límite y el día siguiente
HU-04.5-A1	Entrega 2	Servidor	Programar el cierre automático a los tres días hábiles como «no verificada»
HU-04.5-A2	Entrega 2	Pruebas	Probar el cálculo de días hábiles
HU-05.1-A1	Entrega 2	Servidor	Registrar la solicitud de escalamiento con evidencia obligatoria
HU-05.1-A2	Entrega 2	Interfaz	Agregar el formulario de escalamiento con datos precargados
HU-05.1-A3	Entrega 2	Integración	Notificar al Jefe con folio, diagnóstico y tiempo consumido
HU-05.1-A4	Entrega 2	Pruebas	Probar la falta de evidencia
HU-05.2-A1	Entrega 2	Servidor	Implementar la autorización y el rechazo con justificación, funcional y externo
HU-05.2-A2	Entrega 2	Servidor	Reasignar al Nivel 2 y actualizar prioridad y compromiso
HU-05.2-A3	Entrega 2	Integración	Generar el registro ante la DTIC Central o el proveedor
HU-05.2-A4	Entrega 2	Pruebas	Probar las tres decisiones
HU-05.3-A1	Entrega 3	Servidor	Programar el escalamiento automático a la hora sin respuesta en P1
HU-05.3-A2	Entrega 3	Pruebas	Probar la constancia del escalamiento automático
HU-05.4-A1	Entrega 2	Interfaz	Presentar al Nivel 2 la evidencia y la bitácora del diagnóstico previo
HU-05.4-A2	Entrega 2	Pruebas	Probar que la reasignación conserva la bitácora
HU-06.1-A1	Entrega 2	Diseño	Definir la estructura del artículo: síntoma, causa, procedimiento y verificación

Actividad	Entrega	Tipo	Descripción
HU-06.1-A2	Entrega 2	Servidor	Implementar la búsqueda por categoría y palabra clave con contador de consultas
HU-06.1-A3	Entrega 2	Interfaz	Construir la pantalla de búsqueda y resultados
HU-06.1-A4	Entrega 2	Pruebas	Probar el orden por frecuencia y la constancia en bitácora
HU-06.2-A1	Entrega 2	Servidor	Implementar el borrador de artículo y su publicación tras la validación
HU-06.2-A2	Entrega 2	Servidor	Impedir el cierre si una solución no documentada carece de artículo (RN-12)
HU-06.2-A3	Entrega 2	Interfaz	Construir el editor de artículos
HU-06.2-A4	Entrega 2	Pruebas	Probar el bloqueo de cierre
HU-06.3-A1	Entrega 3	Servidor	Exponer solo artículos validados de acceso público
HU-06.3-A2	Entrega 3	Interfaz	Agregar la búsqueda de autoayuda en el portal
HU-06.3-A3	Entrega 3	Pruebas	Probar que no se exponen borradores
HU-07.3-A1	Entrega 2	Servidor	Implementar reintentos con intervalos crecientes y el aviso interno de respaldo
HU-07.3-A2	Entrega 2	Pruebas	Simular la caída del servicio de correo
HU-09.1-A1	Entrega 2	Diseño	Definir los indicadores y su fórmula de cálculo
HU-09.1-A2	Entrega 2	Servidor	Consolidar los datos del periodo en menos de diez segundos para un mes
HU-09.1-A3	Entrega 2	Interfaz	Construir el tablero con resumen y gráfica de cumplimiento
HU-09.1-A4	Entrega 2	Pruebas	Probar la exactitud de los indicadores y el tiempo de respuesta
HU-09.2-A1	Entrega 2	Servidor	Generar la exportación a PDF y a hoja de cálculo
HU-09.2-A2	Entrega 2	Interfaz	Agregar los botones de exportación
HU-09.2-A3	Entrega 2	Pruebas	Probar el contenido y el formato de los archivos
HU-10.2-A1	Entrega 2	Servidor	Validar que siempre exista un administrador activo
HU-10.2-A2	Entrega 2	Servidor	Exigir la reasignación de solicitudes abiertas antes de la baja
HU-10.2-A3	Entrega 2	Pruebas	Probar los dos bloqueos
HU-11.2-A1	Entrega 2	Servidor	Validar que ninguna categoría activa quede sin técnico habilitado
HU-11.2-A2	Entrega 2	Pruebas	Probar el bloqueo y el mensaje
HU-11.3-A1	Entrega 2	Diseño	Modelar la ficha del activo y la orden de trabajo de mantenimiento
HU-11.3-A2	Entrega 2	Integración	Consultar resguardos en el repositorio patrimonial y marcar «pendiente de regularización»
HU-11.3-A3	Entrega 2	Servidor	Programar el mantenimiento evitando el horario de clases
HU-11.3-A4	Entrega 2	Interfaz	Construir el calendario semestral y la ficha del activo
HU-11.3-A5	Entrega 2	Pruebas	Probar programación, resguardo faltante y conclusión
HU-11.4-A1	Entrega 2	Servidor	Actualizar estado y fecha del próximo mantenimiento al concluir la rutina
HU-11.4-A2	Entrega 2	Servidor	Emitir el acta con las firmas de ejecución y de conformidad
HU-11.4-A3	Entrega 2	Interfaz	Agregar la captura de hallazgos y refacciones
HU-11.4-A4	Entrega 2	Pruebas	Probar la actualización y el contenido del acta
HU-12.1-A1	Entrega 3	Datos y modelo	Calcular exactitud global y por categoría por versión del modelo
HU-12.1-A2	Entrega 3	Interfaz	Construir el panel de supervisión
HU-12.1-A3	Entrega 3	Pruebas	Probar el cálculo con predicciones verificadas
HU-12.2-A1	Entrega 3	Datos y modelo	Implementar el reentrenamiento con el volumen mínimo de ejemplos por categoría
HU-12.2-A2	Entrega 3	Datos y modelo	Comparar la versión nueva con la vigente y conservar la mejor
HU-12.2-A3	Entrega 3	Interfaz	Agregar la orden de reentrenamiento y el aviso de categorías con pocos datos
HU-12.2-A4	Entrega 3	Pruebas	Probar los tres criterios

Tabla 12. Actividades del backlog posterior.

Elaboración propia con base en las Historias de usuario, versión 3.

Referencias

- Boehm, B., & Turner, R. (2003). *Balancing agility and discipline: A guide for the perplexed*. Addison-Wesley.
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The unified software development process*. Addison-Wesley.
- Kruchten, P. (2003). *The Rational Unified Process: An introduction* (3.^a ed.). Addison-Wesley.
- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. Scrum.org.
<https://scrumguides.org/scrum-guide.html>